

A Drift-Aware MLOps Framework for Multi-Model Serving: A Network Intrusion Detection Case Study

Tran Nguyen Viet Hoang

*Faculty of Computer Networks and Communications
University of Information Technology, VNU-HCM
Ho Chi Minh City, Vietnam
23520541@gm.uit.edu.vn*

Bui Ngoc Thai

*Faculty of Computer Networks and Communications
University of Information Technology, VNU-HCM
Ho Chi Minh City, Vietnam
23521412@gm.uit.edu.vn*

Abstract—Machine learning systems deployed in production are affected not only by source-code changes but also by changes in data distributions. This issue is critical for network intrusion detection, where traffic patterns and attack behaviors evolve over time. However, this paper does not focus on proposing a new intrusion detection model. Instead, it presents a baseline design of a drift-aware MLOps framework that supports user-imported tabular classification models and uses Network Intrusion Detection Systems (NIDS) as a case study. The proposed framework separates model governance from model serving through a control-plane and data-plane architecture. It supports model onboarding, dynamic endpoint generation, tenant-aware access control, asynchronous production logging, reference-production drift detection, and adaptation-trigger generation. We formalize the system as a lifecycle over tenants, models, inference requests, production data windows, drift scores, and lifecycle actions. The prototype is implemented with Django, FastAPI, MLflow, PostgreSQL, Redpanda, and Evidently AI. The baseline demonstrates how a model can be uploaded, served as an API, monitored from production logs, and evaluated for Data Drift.

Index Terms—MLOps, Data Drift, multi-model serving, model monitoring, tabular classification, network intrusion detection.

I. INTRODUCTION

Machine learning (ML) models are increasingly deployed as continuously running services rather than offline experimental artifacts. In a production setting, a model receives new input samples over time, generates predictions for downstream users or systems, and depends on monitoring mechanisms to remain trustworthy. Unlike traditional software services, an ML service may degrade even when its source code and infrastructure remain unchanged. One major cause is Data Drift, where the production data distribution differs from the reference distribution used during training, validation, or initial deployment.

This problem is especially relevant in Network Intrusion Detection Systems (NIDS). Network traffic is non-stationary: benign usage patterns change, infrastructure evolves, and attack strategies continuously adapt. A classifier trained on historical flow features may therefore become less reliable when deployed in a new environment or under a new attack distribution. A production-grade NIDS should not only expose a prediction API but also record production data, compare it with reference data, and trigger lifecycle actions when the monitored distribution changes.

Existing MLOps research provides architectural concepts for automating ML development, deployment, and monitoring [1]. Other works discuss MLOps maturity, pipeline platforms, data quality, and continual model maintenance [2], [4], [5]. However, practical systems often treat monitoring as a passive observability layer. In many prototypes, drift reports are generated after deployment, but the monitoring result is not explicitly modeled as a control signal for model lifecycle decisions.

This paper presents a baseline framework that connects multi-model serving with drift-aware monitoring. The goal is to describe a reusable MLOps framework, not to claim a new state-of-the-art NIDS classifier. In the proposed design, NIDS is used as a case study because it naturally exposes the need for drift-aware operation. The framework is intentionally scoped to user-imported tabular classification models packaged in a compatible format such as MLflow pyfunc.

The main contributions of this paper are:

- We propose a tenant-aware and drift-aware MLOps framework that integrates model onboarding, dynamic serving, production logging, drift detection, and lifecycle adaptation triggers.
- We formalize the framework using tenants, model functions, reference data, production windows, feature-level drift indicators, dataset-level drift ratio, and an action-selection function.
- We implement a prototype platform and instantiate it with a Network Intrusion Detection case study to demonstrate the feasibility of the proposed lifecycle.

II. RELATED WORK

A. MLOps Architectures

MLOps extends DevOps principles to ML systems by coordinating data engineering, experimentation, training, deployment, monitoring, and governance. Kreuzberger *et al.* define MLOps as a paradigm for reliably and efficiently developing and maintaining ML systems in production [1]. Their architecture emphasizes components such as data pipelines, training pipelines, model registries, serving infrastructure, and monitoring. This work provides the architectural foundation for our paper, but our focus is narrower: we operationalize

monitoring as a drift-aware lifecycle signal in a multi-model serving platform.

John *et al.* propose a framework and maturity model for MLOps adoption [2]. Their perspective suggests that production ML systems can be evaluated by the degree of automation and integration across the lifecycle. Our framework follows this direction by combining onboarding, serving, logging, and monitoring in a single flow. However, instead of discussing maturity at a general organizational level, we express the lifecycle as a set of model-specific state transitions.

B. MLOps Platforms and Case Studies

Several studies present MLOps platforms through implementation-oriented case studies. Zhou *et al.* discuss a machine learning pipeline platform that connects different stages of model development and deployment [3]. Subramanya *et al.* describe an MLOps workflow and apply it to electricity market forecasting [4]. These works show that case studies are useful for validating MLOps workflows in a concrete domain. Similarly, this paper uses NIDS as a case study, but the main artifact is the reusable serving and monitoring framework rather than the intrusion detection model itself.

C. Data Quality and Drift Monitoring

Production ML quality depends on the data observed after deployment. Renggli *et al.* argue that data quality should be treated as a first-class concern in MLOps [5]. Data Drift detection is one practical mechanism for identifying when production data no longer resembles the reference data. Gama *et al.* survey concept drift adaptation and show that non-stationarity is a central challenge in learning systems [6]. In this paper, Data Drift is used as an operational signal. Because production labels may be delayed or unavailable, the system first compares input distributions and then triggers inspection, retraining, or model replacement workflows when drift is detected.

D. Research Gap

The reviewed literature provides three useful foundations: general MLOps architecture, implementation-oriented MLOps case studies, and drift adaptation theory. The gap addressed in this baseline is the integration of these ideas into a concrete multi-model serving framework where drift detection is represented as a model-specific control signal. The proposed framework therefore bridges platform engineering and drift-aware model lifecycle management.

III. PROBLEM FORMULATION

Let $\mathcal{U} = \{u_1, u_2, \dots, u_N\}$ denote the set of tenants using the platform. Each tenant $u \in \mathcal{U}$ owns a set of registered models:

$$\mathcal{M}_u = \{m_{u,1}, m_{u,2}, \dots, m_{u,K_u}\}. \quad (1)$$

Each model $m_{u,i}$ is represented by an inference function parameterized by θ :

$$\hat{y}_t = f_{\theta}^{u,i}(x_t), \quad (2)$$

where $x_t \in \mathbb{R}^d$ is the input vector at time t , d is the number of input features, and $\hat{y}_t$ is the prediction. For a tabular classification model, $\hat{y}_t$ may be a class label or a probability vector over class labels.

For every inference request, the platform records a production log:

$$z_t = (u, i, x_t, \hat{y}_t, c_t, t), \quad (3)$$

where c_t is the prediction confidence if available. The log stream associated with a tenant-model pair (u, i) forms the basis for monitoring.

Each deployed model has a reference dataset:

$$D_{\text{ref}}^{u,i} = \{x_1, x_2, \dots, x_n\}, \quad (4)$$

which represents the distribution observed during training, validation, or initial acceptance. At monitoring time T , the current production data window is:

$$D_{\text{cur}}^{u,i}(T) = \{x_t \mid T - W \leq t \leq T\}, \quad (5)$$

where W is the monitoring window size. The objective is not only to compute predictions but also to select a lifecycle action:

$$a_T = \pi(m_{u,i}, D_{\text{ref}}^{u,i}, D_{\text{cur}}^{u,i}(T), S_T), \quad (6)$$

where $\pi(\cdot)$ is the adaptation policy and S_T is the current lifecycle state of the model. Possible actions include no operation, alert generation, manual review, retraining trigger, model rollback, or model promotion.

For each feature j , the framework computes a drift score:

$$\delta_j = \Delta(P_{\text{ref}}(x_j), P_{\text{cur}}(x_j)), \quad (7)$$

where $\Delta(\cdot)$ is a statistical distance or hypothesis test between the reference and current feature distributions. A feature-level drift indicator is defined as:

$$I_j = \begin{cases} 1, & \delta_j > \tau_j, \\ 0, & \text{otherwise,} \end{cases} \quad (8)$$

where τ_j is the threshold for feature j . The dataset-level drift ratio is:

$$\rho = \frac{1}{d} \sum_{j=1}^d I_j. \quad (9)$$

The final drift decision is:

$$\text{Drift}(D_{\text{ref}}, D_{\text{cur}}) = \mathbb{K}[\rho \geq \tau_D], \quad (10)$$

where τ_D is the dataset-level threshold. This formulation separates model prediction from monitoring and makes Data Drift an explicit input to lifecycle decisions.

IV. PROPOSED FRAMEWORK

The logical flow of the proposed framework consists of four stages. First, a user uploads a model artifact through the control plane. Second, the data plane exposes the model as a prediction endpoint. Third, each prediction produces an asynchronous log event. Fourth, the monitoring layer periodically reconstructs the current production window and compares it with the reference data. If drift is detected, the adaptation layer creates an alert or triggers a retraining workflow.

A. Control Plane

The control plane manages user-facing and governance operations. It stores tenants, authentication credentials, API keys, model metadata, artifact locations, endpoint URLs, access modes, and model statuses. In the prototype, this layer is implemented using Django. A model record contains the model name, description, owner tenant, artifact URI, access mode, generated endpoint, and lifecycle status.

When a model artifact is uploaded, the control plane validates the package at a structural level and checks for an MLflow `MLmodel` specification. The model is then registered and assigned an endpoint of the form `/models/{model_id}/predict`. This design decouples model ownership and metadata management from online inference.

B. Data Plane

The data plane is implemented as a generic FastAPI model server. Instead of deploying one service per model, the server resolves the model through the path parameter `model_id`. It reads the corresponding model metadata, loads the model artifact through MLflow `pyfunc`, validates the request payload against the expected input format when possible, and returns the prediction result.

Tenant-aware access control is enforced before inference. Public models can be invoked directly, while private models require a valid API key or bearer token. The resolved tenant identifier must match the model owner. This mechanism supports a platform setting where multiple users upload and serve independent models through a shared serving layer.

C. Production Logging Layer

Online inference should not depend synchronously on database writes. Therefore, the framework decouples prediction from logging through a message broker. After producing a prediction, the model server emits a production log event to Redpanda. A consumer service batches log events and persists them to PostgreSQL. Each log stores the tenant identifier, model identifier, timestamp, feature payload, and prediction result.

This design has two advantages. First, it reduces the latency impact of monitoring on the prediction path. Second, it creates a unified production data source that can be queried per tenant and per model for drift detection, audit, and evaluation.

Fig. 1. Model onboarding procedure.

Require: Model package P , metadata H , tenant u
Ensure: Registered model endpoint e

- 1: Validate package extension and archive structure
- 2: Check whether P contains an `MLmodel` specification
- 3: Store P in artifact storage
- 4: Create model record $m_{u,i}$ in the control plane
- 5: Assign lifecycle state $S \leftarrow \text{validated}$
- 6: Generate endpoint $e \leftarrow \text{/models/{i}/predict}$
- 7: Return endpoint e

D. Drift Monitoring Layer

The monitoring layer compares reference data with recent production data. In the prototype, Evidently AI is used to compute feature-level drift and dataset-level drift summaries. Reference data can be stored as object storage artifacts, while production data are reconstructed from PostgreSQL logs. Drift reports can be exported as HTML or JSON artifacts for inspection.

E. Adaptation Layer

The adaptation layer maps monitoring outputs to lifecycle actions. In the baseline implementation, the primary action is a drift alert or workflow trigger. The framework can be extended to support retraining jobs, model evaluation gates, model promotion, rollback, and notification. This distinction is important: the current prototype demonstrates a drift-aware control loop, while fully autonomous retraining and deployment remain future work.

V. DRIFT-AWARE LIFECYCLE ALGORITHM

The model lifecycle can be represented as a state transition system:

$$S \in \{\text{uploaded, validated, deployed, monitored, drifted, reviewing, retraining, promoted, failed}\}. \quad (11)$$

The transition function is:

$$S_{T+1} = \Phi(S_T, m_{u,i}, \rho, \tau_D, a_T), \quad (12)$$

where $\Phi(\cdot)$ updates the model state according to the drift ratio and the selected action. If $\rho < \tau_D$, the model remains in the monitored state. If $\rho \geq \tau_D$, the model is marked as drifted and the system triggers alerting or retraining.

VI. CASE STUDY: NETWORK INTRUSION DETECTION

The NIDS model is used as a case study to instantiate the proposed framework. A network flow is represented as a tabular feature vector $x \in \mathbb{R}^d$ containing statistics such as duration, packet counts, byte rates, and protocol-related attributes. The model predicts whether the observed traffic is benign or belongs to an attack class.

In the current prototype, XGBoost is used as the classification algorithm because it is widely used for tabular data and can be packaged as an MLflow-compatible model. The

Fig. 2. Drift-aware inference and monitoring procedure.

Require: Input x_t , model identifier i , credential q
Ensure: Prediction $\hat{y}_t$ and optional lifecycle action a_T

- 1: Authenticate q and resolve tenant u
- 2: Retrieve model metadata $m_{u,i}$
- 3: Load or reuse cached model $f_{\theta}^{u,i}$
- 4: Compute prediction $\hat{y}_t \leftarrow f_{\theta}^{u,i}(x_t)$
- 5: Emit log event $z_t = (u, i, x_t, \hat{y}_t, t)$
- 6: **if** monitoring condition is satisfied **then**
- 7: Build production window $D_{\text{cur}}^{u,i}(T)$
- 8: **for** each feature $j = 1$ to d **do**
- 9: Compute drift score δ_j
- 10: Compute indicator I_j
- 11: **end for**
- 12: Compute drift ratio ρ
- 13: Select action $a_T \leftarrow \pi(m_{u,i}, D_{\text{ref}}, D_{\text{cur}}, S_T)$
- 14: **if** $\rho \geq \tau_D$ **then**
- 15: Trigger alert, review, or retraining workflow
- 16: **end if**
- 17: **end if**
- 18: Return $\hat{y}_t$

project contains multiple trained model versions, including a binary classifier and an extended classifier that includes additional attack behavior. These models are treated as imported artifacts rather than hard-coded services. This allows the case study to demonstrate the general framework behavior: model registration, endpoint invocation, production logging, and drift monitoring.

NIDS is an appropriate case study for drift-aware MLOps for three reasons. First, network environments change over time, causing benign traffic distributions to shift. Second, attack patterns evolve, making historical training data incomplete. Third, labeled production data are often delayed or unavailable, so input distribution monitoring can provide an early warning signal before supervised performance metrics are available.

VII. PROTOTYPE IMPLEMENTATION

Table I summarizes the prototype modules and their roles.

TABLE I
PROTOTYPE IMPLEMENTATION MODULES

Module	Role
Django control plane	Users, tenants, API keys, model metadata
FastAPI model server	Dynamic endpoint and MLflow model loading
Redpanda	Asynchronous production log transport
PostgreSQL	Metadata and production log persistence
Evidently AI	Reference-production drift detection
MLflow	Model packaging and pyfunc loading
Kubernetes/K3s	Container orchestration and deployment
Argo CD	GitOps-based synchronization

The implemented flow is as follows. A user authenticates to the control plane and uploads an MLflow-compatible model package. The control plane creates a model record and returns an endpoint. The user sends prediction requests to the generated endpoint with an API key or token. The model server resolves the model, performs inference, and publishes

production logs to Redpanda. A consumer persists the logs to PostgreSQL. The drift service later retrieves production logs for a specific tenant-model pair and compares them with reference data using Evidently AI.

Table II describes the current baseline coverage. The framework already supports core platform operations, while several production-hardening features remain future work.

TABLE II
BASELINE FEATURE COVERAGE

Capability	Status
User authentication	Implemented
API key management	Implemented
Model artifact upload	Implemented
Dynamic endpoint generation	Implemented
MLflow pyfunc loading	Implemented
Production logging	Implemented
Reference-production drift report	Partially implemented
Automated retraining and promotion	Future work
Resource quota and billing	Future work

VIII. EVALUATION DESIGN

Because this paper is a baseline, the evaluation focuses on feasibility and system behavior rather than claiming superior NIDS accuracy. The following experimental design is proposed.

A. Model Onboarding Validation

The first experiment verifies whether the platform can receive a model package, check for the required MLflow metadata, store the artifact, create model metadata, and generate a usable endpoint. The metrics include onboarding success rate, validation failure cases, and time required to register a model.

B. Inference Serving Validation

The second experiment evaluates whether the generated endpoint can process tabular records and return predictions. JSON records and CSV-derived rows can be used as input. The metrics include request success rate, average latency, and model loading behavior with and without cache reuse.

C. Production Logging Validation

The third experiment verifies that each inference request creates a production log event and that the event is eventually persisted to PostgreSQL. The expected result is that logs can be queried by tenant and model identifier. This experiment validates the isolation needed for multi-model monitoring.

D. Drift Detection Validation

The fourth experiment evaluates whether the drift service detects distribution changes between reference data and production data. A synthetic drift scenario can be created by changing the proportion of selected traffic classes or shifting selected numerical features. The reported outputs should include the number of drifted features, the drift ratio ρ , the dataset-level drift decision, and the generated drift report.

IX. DISCUSSION

The proposed framework reframes MLOps monitoring as part of an adaptive control loop. The model server does not only return predictions; it also emits production evidence. The monitoring layer does not only generate reports; it produces a signal that can affect model lifecycle decisions. This design is useful for a SaaS or PaaS-style environment where different users upload different models but still need a common operational pipeline.

The current scope is intentionally limited. First, the system supports tabular classification models packaged in an MLflow-compatible format. Image, text, time-series, and large language models require additional schema validation and monitoring metrics. Second, arbitrary user-uploaded model artifacts introduce security risks. A production system must isolate execution, validate dependencies, restrict archive extraction, and enforce resource quotas. Third, fully autonomous retraining and promotion require reliable evaluation gates, rollback policies, and human approval mechanisms. Therefore, this baseline treats drift detection as an adaptation trigger rather than an unconditional automatic redeployment decision.

X. CONCLUSION

This paper presented a baseline design for a drift-aware MLOps framework for multi-model serving. The framework integrates tenant-aware model onboarding, dynamic endpoint generation, access control, asynchronous production logging, and reference-production drift monitoring. The system was formalized using tenants, model functions, production data windows, drift indicators, drift ratio, lifecycle states, and adaptation actions. A Network Intrusion Detection model was used as a case study to demonstrate the applicability of the framework to a non-stationary security domain. Future work will focus on hardening artifact execution, storing drift results in the control plane, implementing retraining and model promotion workflows, and extending the framework to additional model types.

REFERENCES

- [1] M. Kreuzberger, N. Kuhl, and S. Hirschl, "Machine Learning Operations (MLOps): Overview, Definition, and Architecture," *IEEE Access*, vol. 11, pp. 31866–31879, 2023.
- [2] M. M. John, H. H. Olsson, and J. Bosch, "Towards MLOps: A Framework and Maturity Model," in *Proc. 47th Euromicro Conference on Software Engineering and Advanced Applications*, 2021, pp. 1–8.
- [3] Y. Zhou, Y. Yu, and B. Ding, "Towards MLOps: A Case Study of ML Pipeline Platform," in *Proc. International Conference on Artificial Intelligence and Computer Engineering*, 2020, pp. 494–500.
- [4] S. G. Subramanya, S. Shetty, V. Patil, S. K. Gad, and S. N. Bhat, "From DevOps to MLOps: Overview and Application to Electricity Market Forecasting," *Applied Sciences*, vol. 12, no. 19, 2022.
- [5] C. Renggli, B. Karlaš, B. Ding, F. Liu, K. Schawinski, W. Wu, and C. Zhang, "A Data Quality-Driven View of MLOps," *arXiv preprint arXiv:2102.07750*, 2021.
- [6] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A Survey on Concept Drift Adaptation," *ACM Computing Surveys*, vol. 46, no. 4, pp. 1–37, 2014.
- [7] M. Zaharia et al., "Accelerating the Machine Learning Lifecycle with MLflow," *IEEE Data Engineering Bulletin*, vol. 41, no. 4, pp. 39–45, 2018.

- [8] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [9] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in *Proc. 4th International Conference on Information Systems Security and Privacy*, 2018, pp. 108–116.
- [10] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.